Mule Ring Hunter: Team Helper Guide

Oct 16, 2026 · @Dawg

Start here

This guide is the one place to look during the datathon. It follows our six-step pipeline: each step has its own section explaining what it does, what goes in and out, how to build it, links, and how to know it works.

How to use it:

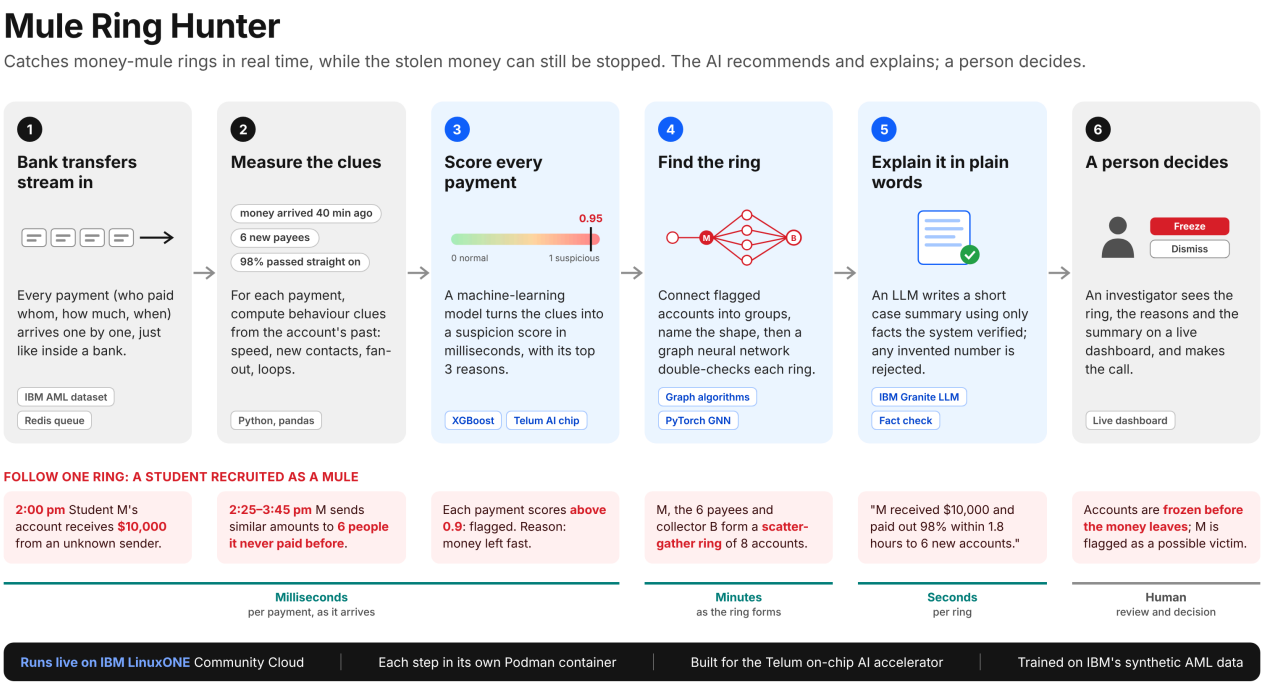
1. Read this section and **Team-wide rules** (everyone, 10 minutes).
2. Read **The dataset** (everyone; Pair 1 reads it twice).
3. Jump to the step sections your pair owns (table below).
4. Before you connect your step to the next one, check **Handoffs between steps**. It defines exactly what each step passes on, so pairs can work in parallel without breaking each other.
5. Stuck for more than 20 minutes? Check **When you're stuck**, then ask in the Discord IBM expert channel.

The whole system in one sentence: **payments stream in, each one is scored in milliseconds, suspicious ones are grouped into rings within minutes, Granite writes a fact-checked summary in seconds, and a person decides.**

Step	What happens	Main tools	Owner
1. Bank transfers stream in	Replay the dataset one payment at a time, in time order, into a queue	Redis Streams, Python	Pair 3
2. Measure the clues	For each payment, compute behaviour clues from the account's past only	pandas, Python	Pair 1 (simple clues), Pair 2 (time-window clues)
3. Score every payment	XGBoost turns the clues into a 0 to 1 suspicion score plus its top 3 reasons	XGBoost, Telum via Snap ML	Pair 2 (model), Pair 3 (Telum)

Step	What happens	Main tools	Owner
4. Find the ring	Connect flagged accounts into groups, name the shape, re-check with a GNN	NetworkX-style graph code, PyTorch	Pair 2
5. Explain it in plain words	Granite writes a case summary from verified facts; a fact check rejects invented numbers	IBM Granite, Python	Pair 1
6. A person decides	Live page shows rings, reasons and summary; investigator clicks Freeze or Dismiss	Streamlit (or Flask)	Pair 3
Platform	Everything runs in containers on LinuxONE, with screenshots as evidence	Docker or Podman, LinuxONE	Pair 3
Proof and pitch	Evaluation numbers, the 3-minute video, the submission	scikit-learn metrics	Pair 1

The pipeline image below is also our submission flowchart; keep it open while you work.



Quick links

Every link here was opened and checked on October 16. Libraries we use must also be linked in the submission README.

Event and platform

Link	What it's for
Hack2Skill event page	Where we submit
How to submit (video)	The organizers' walkthrough of the submission form
Discord	Announcements, help desk, IBM expert channel on event day
LinuxONE Community Cloud login	Our VM dashboard
LinuxONE Fast Start guides	Guides and tutorials for the VM
LinuxONE Jupyter Lab image	The container with PyTorch, XGBoost and scikit-learn already built for s390x

Data and background

Link	What it's for
IBM AML dataset on Kaggle	Download HI-Small_Trans.csv and HI-Small_Patterns.txt
The paper behind the dataset	How IBM generated the data and the laundering patterns (Altman et al., NeurIPS 2023)
IBM Multi-GNN repo	IBM's research GNNs on this data. Read for ideas only; don't copy code
Cifas: money mules	Statistic for the pitch: 21 to 30 year olds were 42% of UK money mule activity in 2020 (17,157 cases)
FBI: money mules	"Criminals often target students, those looking for work..."

Libraries, by step

Step	Link	What to read
1	Redis Streams guide	XADD, XREADGROUP, consumer groups, XACK
1	redis-py docs	The Python client
2	pandas user guide	Reading CSV, sorting, grouping, saving parquet
3	XGBoost Python intro	Training, early stopping, prediction
3	XGBoost model saving	Why JSON, and how
3	precision_recall_curve	Picking thresholds
3	IBM Z Accelerated for Snap ML	Running tree models on Telum
3	Snap ML boosting API	`import_model` for loading our XGBoost JSON
4	NetworkX components	Weakly connected components, to check our union-find

Step	Link	What to read
4	NetworkX Louvain	Splitting rings that are too big
4	PyTorch: Learn the Basics	Tensors, models, training loops, for the GNN
4	PyTorch ONNX export	Only if the GNN is trained off the VM, or for zDLC
4	IBM zDLC	Optional: compile the ONNX GNN for Telum
5	IBM Granite on Hugging Face	The Granite model family
5	watsonx.ai Python SDK: ModelInference	`ModelInference(...).chat(...)` to call Granite
5	watsonx.ai REST API	The text and chat endpoints, if calling over plain HTTPS
5	IBM Cloud IAM token API	Turning an API key into an access token
6	st.graphviz_chart	Drawing the ring picture
6	Flask docs	Fallback if Streamlit won't install
6	Python graphviz package	Building ring pictures (needs the Graphviz system package too)
Platform	Docker Compose	Running all containers from one file
Platform	podman-compose	The same, if the VM uses Podman

The dataset

We use IBM's synthetic anti-money-laundering data, the **HI-Small** version: 10 days of bank transfers (September 1 to 10, 2022): about 5 million transfers between 515,000 accounts, of which about 5,100 are laundering (1 in 981), according to the Kaggle page. It is synthetic (generated by IBM's simulator), so there is no real customer data and no privacy problem, which is worth saying in the pitch.

Where to get it: the Kaggle dataset page (link in **Quick links**). We need only two files:

File	What it is	Who uses it
HI-Small_Trans.csv	Every transfer, one per row, with a label saying whether it is laundering	Everyone
HI-Small_Patterns.txt	The known laundering attempts, grouped into blocks, each with its shape name	Pair 1 for evaluation only; never as a model input

Download it once onto the VM so everyone shares one copy:

```
mkdir -p ~/shared/data && cd ~/shared/data
kaggle datasets download -d ealtman2019/ibm-transactions-for-anti-money-laundering-aml -f HI-Small_Trans.csv
kaggle datasets download -d ealtman2019/ibm-transactions-for-anti-money-laundering-aml -f HI-Small_Patterns.txt
unzip -o '*.zip'
```

The `~/shared` folder is mounted inside the Jupyter container as `/home/jovyan/shared`, so notebooks read the files from `/home/jovyan/shared/data/`.

The columns in HI-Small_Trans.csv

Column (as in the file)	Example	What it means	Rename to
Timestamp	2022/09/01 00:20	When the transfer happened, to the minute	ts
From Bank	10	Sender's bank number	from_bank
Account	8000EBD30	Sender's account at that bank	from_acct_raw
To Bank	10	Receiver's bank number	to_bank
Account (second one)	8000EBD30	Receiver's account; pandas names it `Account.1`	to_acct_raw
Amount Received	3697.34	What arrived, in the receiving currency	amt_recv

Column (as in the file)	Example	What it means	Rename to
Receiving Currency	US Dollar	Currency that arrived	ccy_recv
Amount Paid	3697.34	What left, in the paying currency	amt_paid
Payment Currency	US Dollar	Currency that left	ccy_paid
Payment Format	Reinvestment	How it was paid: ACH, Cheque, Credit Card, Wire, Cash, Bitcoin, Reinvestment	fmt
Is Laundering	0	The answer: 1 = laundering, 0 = normal	label

Rules for cleaning (Pair 1 does this first; everyone uses the result)

1. **Read with pandas, then rename the columns** as in the last column above. The file has two columns called "Account"; pandas automatically calls the second one `Account.1`.
2. **Build one account ID** per account: bank number without leading zeros, an underscore, then the account. Example: bank `010`, account `8000EBD30` becomes `10_8000EBD30`. The same account text can exist at two banks, so the bank must be part of the ID. Use exactly this rule everywhere, including when reading the patterns file.
3. **Parse the time** with the format `%Y/%m/%d %H:%M`. Many transfers share the same minute, so sort by time and keep the original row order as a tie-breaker (stable sort), then give every row a `txn_id` = its position after sorting.
4. **Make one comparable amount.** Currencies differ, so build an exchange-rate table: for each currency, look at rows where one side is US Dollar and take the median ratio. Then `amount_usd` = amount paid times that currency's rate. Spot-check a few rows by hand.
5. **Flag self-transfers** (sender equals receiver). They are mostly Reinvestment. Keep them, but mark them, because they are not real money movement between people.
6. **Save the clean table** as `clean.parquet` (or CSV if parquet fails) in `~/shared/data/`.

The patterns file

Each laundering attempt is a block: a line starting `BEGIN LAUNDERING ATTEMPT -` followed by the shape name (for example FAN-OUT, FAN-IN, CYCLE, SCATTER-GATHER, GATHER-SCATTER, BIPARTITE, STACK or RANDOM), then the attempt's transfers in the same comma format as the main file, then a line starting `END LAUNDERING ATTEMPT`. Parse it into a table with one row per transfer plus `attempt_id` and `shape`. Some shape lines carry extra text after a colon (such as a maximum degree); keep only the word before the colon.

Not every laundering transfer in the main file belongs to a block, so the blocks are a partial answer key for rings, not a full one.

Things that will trip you up

- **Accuracy is useless here.** A model that says "never laundering" is 99.9% accurate. Use precision, recall and F1 (see **Glossary**).
- **The label is never an input.** `Is Laundering` and anything from the patterns file must never become a feature.
- **Never shuffle.** Split by time (see **Team-wide rules**). Shuffling lets the model see the future.
- **Check the date range yourself** after loading (`df.ts.min()`, `df.ts.max()`) and how many laundering rows each split has. Write both numbers in the team chat; the pitch needs them.

Team-wide rules

These rules apply to every pair. Most bugs in projects like this come from breaking one of them, so read them before writing any code.

Competition rules

- **All code is written during the event** (Saturday 8 AM to Sunday 9 AM, Chicago time). Practice projects, old notebooks and GitHub repos are for learning only. Don't copy code from them, including the IBM reference GNN repo.
- **Libraries are fine, but list them.** The submission must link every open-source library we use. Add each one to the `Libraries` list in the README the moment you `import` it.

- **Finish the two provided labs first.** The organizers say the two AI/ML labs (multi-model predictive AI, and predictive plus generative AI) are mandatory before building. Whoever has finished them confirms in the team chat.
- **Commit to GitHub often** (at least every hour per pair). The commit times prove the work was done during the event.

The time split

We split the data by time, never at random, because in real life the model is trained on the past and used on the future.

Part	Which transfers	Used for
Train	First 60% of transfers by time	Fitting the model
Validation	Next 20%	Choosing settings and thresholds
Test	Last 20%	The final numbers in the pitch, touched once

Pair 1 computes the two cut-off timestamps once and posts them in the chat; everyone uses those exact timestamps. The replay in step 1 streams the **test** part, so the live demo shows transfers the model never saw.

No peeking at the future

A clue for a transfer may only use transfers that happened **before** it. Example: "how many people did this account pay in the last 24 hours" must count only earlier payments, not the full day. If a clue would be different when computed live (one transfer at a time) than in a notebook (the whole file at once), it is wrong. This mismatch is called train/serve skew, and it makes great notebook numbers that fall apart in the demo.

Thresholds, chosen on validation only

- **Alert threshold:** the score above which one transfer is flagged. Pick the one with the best F1 on the validation part.
- **Ring threshold:** the score above which a transfer may join a ring. Pick the lowest score where precision on validation is at least 0.25 (1 in 4 flagged transfers really is laundering). Lower than the alert threshold on purpose: rings need more edges.
- Save both in `thresholds.json` next to the model so every service reads the same numbers.

Saving models on LinuxONE

LinuxONE stores numbers in the opposite byte order to laptops (big-endian). A model pickled on a Mac can load wrongly or not at all.

- XGBoost: save with `model.save_model("scorer.json")`, always JSON.
- PyTorch: train on the VM inside the Jupyter container if you can. If you train on a laptop, export to ONNX and load that.
- Never use `pickle` or `joblib` to move models between machines.

Shared names

Thing	Name everyone uses
Account ID	`<bank without leading zeros>_<account>`, e.g. `10_8000EBD30`
Transfer ID	`txn_id`, the row position after time sorting
Data folder on the VM	`~/shared/data/` (inside Jupyter: `/home/jovyan/shared/data/`)
Models folder	`~/shared/models/`
Times	ISO text, `2022-09-01T00:20:00`

Step 1: Bank transfers stream in

Owner: Pair 3. This step turns a file into a live feed: a small program (the replayer) reads the test part of the clean data in time order and pushes one transfer at a time into a Redis queue. Everything after it reacts to messages as they arrive, the way a bank system would.

In: `clean.parquet`, test part only. **Out:** messages on the Redis stream `txns`.

What a Redis Stream is

A Redis Stream is an append-only list of messages that several programs can read. The replayer adds messages with `XADD`; the scorer reads them with `XREADGROUP`. A **consumer group** remembers which messages each reader has handled, so you can run two scorer

containers side by side and Redis splits the work between them. That is our scalability story for the judges.

The redis-py calls you need, nothing more:

```
import redis
r = redis.Redis(host="redis", port=6379, decode_responses=True)   # "redis" =
the container name
r.xadd("txns", {"txn_id": "812345", "amount_usd": "9800.0"})          #
values are strings
r.xgroup_create("txns", "scorers", id="0", mkstream=True)             # once;
ignore 'already exists'
msgs = r.xreadgroup("scorers", "scorer-1", {"txns": ">"}, count=100,
block=1000)
r.xack("txns", "scorers", msg_id)                                      #
after handling a message
```

How to build it

1. **Start Redis** as a container (see **LinuxONE platform**) and check `redis-cli ping` answers `PONG`.
2. **Load the test part** of `clean.parquet`, sorted by `ts` then `txn_id`.
3. **Send each row** with `XADD` using the field names in **Handoffs between steps**. Add `sent_at` = the real clock time when sent, so latency can be measured later.
4. **Control the speed.** The data spans days; the demo lasts minutes. Use a speed-up factor: `SPEED = 600` means 10 minutes of data time pass in 1 real second. Sleep between messages by (gap in data time) / SPEED. Make SPEED a setting, and add a `--start` option so the demo can begin just before a known ring.
5. **Pick demo moments in advance.** Pair 1 finds 2 or 3 laundering attempts in the test part with clear shapes (one scatter-gather, one fan-out). Start the replay a few data-hours before them.

Done when

- `redis-cli XLEN txns` grows while the replayer runs.
- `redis-cli XRANGE txns - + COUNT 2` shows the right fields.
- Stopping and restarting the replayer does not crash anything downstream.

Common mistakes

- Sending NaN amounts or numbers as Python objects; Redis stores text, so convert everything with `str()` and parse it back on the other side.
- Forgetting `block=` in `XREADGROUP`, which makes the reader spin and use 100% CPU.
- Pointing to `localhost` inside a container. Inside Docker Compose, use the service name (`redis`).

Step 2: Measure the clues

Owners: Pair 1 (simple clues), Pair 2 (time-window clues). A model can't read "suspicious"; it reads numbers. This step turns each transfer into about 20 numbers (clues, also called features) describing the transfer and how both accounts have behaved **before** it.

In: one transfer (a row or a stream message). **Out:** the same transfer plus its clue values.

The one design decision that matters

Write the clue code **once**, as a class that takes transfers one at a time in time order and keeps a small memory per account (for example, its transfers in the last 24 hours). Then use that same class in two places:

- **Offline:** loop over the whole clean file in time order to build the training table (expect several minutes for 5 million rows; run it once and save `features.parquet`).
- **Live:** inside the scorer, on each message from the stream.

Same code in both places means the clues are identical, so there's no train/serve skew. The rule inside the class: **compute the clues for a transfer first, then add the transfer to the account's memory.** That order is what guarantees "past only".

Simple clues (Pair 1): need only the transfer itself

Clue	How to compute	Why it might matter
log_amount	log10(1 + amount_usd)	Laundering moves larger sums; the log stops huge amounts dominating
is_round	amount_usd is a whole multiple of 100	Criminals often move round numbers

Clue	How to compute	Why it might matter
fmt_*	One 0/1 column per payment format	Some formats (Cash, Wire, Bitcoin) behave differently
cross_currency	Paid and received currencies differ	Currency hops hide a trail
cross_bank	Sender's and receiver's banks differ	Rings spread across banks
self_transfer	Sender equals receiver	Mostly harmless reinvestment
hour, weekday	From the timestamp	Unusual timing

Time-window clues (Pair 2): need each account's past

Clue	Meaning	Why it might matter
snd_in_sum_24h	Money that arrived at the sender in the past 24 h	A mule first receives
snd_out_sum_24h	Money the sender sent in the past 24 h	then sends on
pass_through	snd_out_sum_24h / snd_in_sum_24h, capped at 5	"98% passed straight on" is close to 1
mins_since_snd_last_in	Minutes since money last arrived at the sender	"Money arrived 40 min ago"
new_payee	1 if the sender never paid this receiver before	Mules pay strangers
snd_payees_24h	Distinct people the sender paid in 24 h	Fan-out
rcv_payers_24h	Distinct people who paid the receiver in 24 h	Fan-in, a collector
snd_out_count_1h	Transfers the sender made in the past hour	Bursts
amt_vs_snd_avg	This amount / the sender's past average amount	Sudden change of habit

Clue	Meaning	Why it might matter
snd_age_days	Days since the sender first appeared	New accounts are riskier

The clue names in the pipeline image (money arrived 40 min ago, 6 new payees, 98% passed straight on) are `mins_since_snd_last_in`, `snd_payees_24h` with `new_payee`, and `pass_through`. Use them in the demo.

How to keep it fast

- Per account, keep a `collections.deque` of (time, amount, other account) and drop entries older than 24 h from the left before computing.
- Keep a `set` of every payee each sender has ever paid, for `new_payee`.
- Skip accounts with more than 300 counterparties in the windows (bank-like hubs); cap their deque length so one huge account can't slow everything.

Done when

- `features.parquet` has one row per transfer, the `txn_id`, the label, and every clue, with no NaN (fill missing with 0 or -1 and say which).
- **Leakage test:** take 1,000 transfers, compute their clues live one by one through the class, compare to the offline table. They must match exactly.
- Each clue's average differs between laundering and normal rows in a sensible direction (Pair 1 makes this table; it is a good pitch slide).

Step 3: Score every payment

Owners: Pair 2 (the model), Pair 3 (the scorer container and Telum). XGBoost reads a transfer's clues and returns a score from 0 (normal) to 1 (suspicious), plus the three clues that pushed the score up most. This is the fast filter: every transfer goes through it in milliseconds.

In: `features.parquet` for training; clue values for one transfer when live. **Out:** `scorer.json`, `thresholds.json`, and live messages on the `scored` stream.

Why XGBoost

XGBoost builds hundreds of small decision trees, each one fixing the mistakes of the ones before. It's the standard choice for tables of numbers like ours: it trains in minutes on a CPU, handles clues on very different scales without tuning, and can say which clues drove each score, which the investigator page and Granite both need.

Training, step by step

1. Load `features.parquet`; split by Pair 1's two cut-off timestamps into train, validation and test.
2. **Handle the imbalance:** set `scale_pos_weight` = (normal rows in train) / (laundering rows in train). This makes each laundering row count as much as all the normal rows together.
3. Train with early stopping on validation:

```
from xgboost import XGBClassifier
model = XGBClassifier(n_estimators=2000, learning_rate=0.05, max_depth=6,
                      subsample=0.8, colsample_bytree=0.8,
tree_method="hist",
                      scale_pos_weight=spw, eval_metric="aucpr",
early_stopping_rounds=100)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=100)
model.save_model("scorer.json")
```

4. Pick the two thresholds on validation (see **Team-wide rules**), using `precision_recall_curve` from scikit-learn. Save them in `thresholds.json`.
5. Report test precision, recall, F1 and PR-AUC **once**, at the end. Also report a baseline ("flag every transfer above $10,000") so the judges see what the model adds.

The three reasons

XGBoost can split each score into one contribution per clue (these are SHAP values):

```
import xgboost as xgb
contribs = model.get_booster().predict(xgb.DMatrix(X_one_row),
pred_contribs=True)[0]
# one number per clue, plus a last "bias" column; the biggest positives are
the top reasons
```

Keep the 3 clues with the largest positive contribution and turn each into a phrase with the clue's value, such as "money left 40 minutes after it arrived". Pair 1 writes these phrases; they also feed Granite.

The live scorer (Pair 3 wires it, Pair 2 checks it)

A loop that: reads a batch of messages from `txns` with `XREADGROUP`, runs each through the clue class from step 2, scores it, writes the result to the `scored` stream (fields in **Handoffs between steps**), then acknowledges the message. Record `scored_at` so latency = `scored_at - sent_at`. Report the 50th and 99th percentile latency (p50, p99).

Telum (Pair 3, after the scorer works)

If `grep -o nnpa /proc/cpuinfo` printed `nnpa`, the VM has the Telum on-chip AI accelerator. IBM's Snap ML library can import a trained XGBoost JSON model with import_model(..., input_type="xgb_json") and runs it on Telum automatically when the accelerator is present. IBM's ready-made Snap ML container needs IBM registry credentials, so ask on Discord at hour 0; otherwise try installing Snap ML with pip. The plan: load `scorer.json` into Snap ML, score the same 10,000 test transfers with plain XGBoost and with Snap ML, and compare time and results. A table "same scores, X times faster on Telum" is a strong technical slide. If Snap ML won't install or `nnpa` is missing, say so in the video and show the plain CPU latency instead; don't sink more than 2 hours here.

Done when

- Validation PR-AUC is clearly above the laundering rate (about 0.001); it should be many times higher. Write down the exact number; it goes in the pitch.
- The live scorer keeps up with the replay, with p99 latency under 50 ms per transfer.
- Feature importance makes sense (pass-through and new-payee clues should rank high). If the top clue is something odd like `hour`, check for leakage.

Step 4: Find the ring

Owner: Pair 2. One suspicious transfer is a hint; a group of accounts passing money between each other is a case. This step connects flagged transfers into groups (rings), names each ring's shape, and asks a graph neural network (GNN) for a second opinion that looks at the whole neighbourhood, not just one transfer.

In: messages on the `scored` stream. **Out:** new or updated rings on the `rings` stream.

Part A: grouping (must-have, build first)

Think of accounts as dots and flagged transfers as arrows between them. A ring is a set of dots joined by arrows.

1. **Keep a window.** Hold every transfer whose score is at or above the **ring threshold**, from the last 24 hours of data time. Drop older ones.
2. **Every 10 data-minutes, regroup.** Use union-find (also called disjoint-set): start with each account alone, and for every kept arrow, merge the sender's and receiver's groups. What's left are connected groups. This is the same as "weakly connected components" in NetworkX, but union-find is a few lines of plain Python and very fast.
3. **Ignore hubs.** Leave out accounts with more than 300 counterparties; they are bank-like accounts that would glue unrelated rings together.
4. **Keep sensible sizes.** Keep groups of 3 to 50 accounts. If a group is bigger than 50, split it with Louvain community detection (`louvain_communities` in NetworkX) and keep the pieces that are 3 to 50.
5. **Keep ring IDs stable.** A ring grows as new transfers arrive. If a new group shares at least half its accounts with an existing ring, it is the same ring: keep its `ring_id` and update it. Otherwise create a new ID.
6. **Score the ring** as the average score of its transfers, and publish it to `rings` when it is new or has changed.

Naming the shape

The shape helps the investigator and Granite explain what is going on. Simple rules over the ring's arrows are enough:

Shape	Rule of thumb	Picture in words
Fan-out	One account sends to 3 or more others in the ring	One mule spreads money out
Fan-in	3 or more accounts send to one account	A collector gathers it
Scatter-gather	A sends to several middle accounts, which all send to B	Our demo story: student M to 6 payees to collector B
Gather-scatter	Several send to one hub, which then sends to several	A middle-man

Shape	Rule of thumb	Picture in words
Cycle	The arrows form a loop back to the start	Money goes round and returns
Bipartite	Two groups, arrows only from one group to the other	Layering between two sets of mules
Stack	Two or more bipartite layers in a row	Deeper layering
Other	None of the above	Still shown, just unnamed

Check the rules in order from most specific (scatter-gather, cycle) to least (fan-out, fan-in). Pair 1 can measure how often our name matches the patterns file's name.

Part B: the GNN second opinion (must-have, keep it small, start by hour 10)

Why add it: XGBoost judges one transfer using the two accounts' own history. It can't see that the receiver's receiver sends everything to the same collector. A GNN passes information along the arrows, so after 2 layers each transfer "knows" about accounts two steps away. That is exactly the ring shape.

The design, in plain PyTorch (no extra graph library, since those may not install on s390x):

- **Graph:** dots = accounts, arrows = transfers. Each arrow carries its clue values from step 2; each dot carries simple numbers (how many transfers in and out, total amounts in and out, as logs).
- **One layer:** every account collects messages from its incoming arrows and, separately, from its outgoing arrows (direction matters: receiving and sending mean different things), then updates its own numbers. The sum-over-arrows is one tensor call, `index_add_`.
- **Two layers** give 2-hop context.
- **Output:** for each arrow, a small network reads [sender's numbers, receiver's numbers, arrow's clues] and outputs a laundering probability.
- **Training:** to keep it fast on a CPU, train only on the candidate part of the graph: transfers XGBoost already scores above a low cut-off, plus their direct neighbours. Use `BCEWithLogitsLoss` with `pos_weight` for the imbalance. Train on the train part, choose its threshold on validation.

```
# the core of message passing: add each arrow's message into its receiving
account
agg = torch.zeros(num_accounts, dim).index_add_(0, dst_index, messages)
```

Live use: when a ring is published, the ring service runs the GNN on the ring's transfers plus their neighbours and adds `gnn_score` (the average) to the ring. The page shows both scores side by side.

Success is honest numbers, not winning. Report the GNN's precision and recall on the test part next to XGBoost's, on the same transfers. If it doesn't beat XGBoost, that comparison still counts as hybrid AI done properly.

Done when

- During a replay, rings appear on the `rings` stream within a few data-hours of a pattern starting.
- At least one demo ring is found with the right shape name.
- The GNN runs inside the ring service container on the VM, and its numbers are in the results table.

Step 5: Explain it in plain words

Owner: Pair 1. When a ring appears, IBM's Granite language model writes a short case summary an investigator can read in 20 seconds. Granite only gets facts our system already verified, and a fact check rejects any reply containing a number or account that isn't in those facts. If the check fails, a fill-in-the-blanks template is used instead, so a case always has a correct summary.

In: a ring from the `rings` stream. **Out:** a case on the `cases` stream.

This step is what the judges mean by "predictive models paired with LLM reasoning": XGBoost and the GNN decide what is suspicious; Granite explains it. Granite never decides.

How to reach Granite

Try these in order and stop at the first that works:

1. **The way the organizers' "Predictive AI + Generative AI" lab does it.** That lab is mandatory anyway, so copy its connection method (not its code).

2. **watsonx.ai** from the VM, with an API key, project ID and region URL from the organizers. Use the `ibm-watsonx-ai` Python package or plain HTTPS requests. Keep the key in an `.env` file that is never committed.
3. **Last resort:** Granite through Ollama on a laptop, with the VM calling it. This weakens the "runs on LinuxONE" story, so say it honestly in the video.

Build the facts pack first (hours 8 to 12)

For each ring, collect everything worth saying into one small JSON object. Every number is computed by our code, never by Granite:

```
{"ring_id": "R-0042", "shape": "scatter-gather", "n_accounts": 8,
 "key_account": "10_8000EBD30", "received_usd": 10000, "passed_on_pct": 98,
 "hours_to_pass_on": 1.8, "new_payees": 6, "collector": "27_80A1C4F20",
 "first_seen": "2022-09-08T14:00:00", "xgb_score": 0.93, "gnn_score": 0.88,
 "top_reasons": ["money left 40 minutes after it arrived", "6 payees never
paid before", "98% passed straight on"]}
```

From the same facts, write the **template summary** with a Python f-string, such as: "Account 10_8000EBD30 received $10,000 and passed on 98% within 1.8 hours to 6 new accounts, ending at 27_80A1C4F20 (scatter-gather ring of 8 accounts)." The template is the safety net, so finish it before touching Granite.

Then the Granite case writer (hours 10 to 16)

- **Instructions to Granite:** you write case summaries for bank fraud investigators; use only the facts given; never invent numbers, names or account IDs; treat the first account as a possible victim who may have been recruited; reply only with JSON in the given format.
- **Fixed reply format:** `summary` (2 sentences), `what_happened`, `why_suspicious`, `possible_victim`, `suggested_next_step`.
- **Low randomness:** temperature 0 or close to it, and a token limit of about 300.

The fact check (the part judges will ask about)

1. The reply must parse as JSON and contain every required key.
2. Pull out every number in the text (strip `$`, `,` and `%`). Each one must match a number in the facts pack, allowing for rounding (10,000 may appear as 10k; 1.8 may appear as 1.8 or 2).

3. Pull out every account-ID-looking token (digits, underscore, letters and digits). Each must be in the ring.
4. Any failure: use the template, and record `source = "template"` with the reason. Success: `source = "granite"`.

Count how often Granite passes the check over the demo replay. "Granite passed the fact check in X of Y cases; the rest fell back to the template" is an honest, impressive line for the pitch.

Keep it off the fast path

Granite takes seconds. Run the case writer as its own container that reads `rings` and writes `cases`, so payment scoring never waits for it.

Done when

- Every ring gets a case, either from Granite or the template.
- A deliberately wrong reply (edit one number by hand in a test) is caught by the fact check.
- The page shows which source wrote each summary.

Step 6: A person decides

Owner: Pair 3. The investigator page is where everything comes together and what the video shows most. It lists live rings, draws each ring as a picture, shows the reasons and the summary, and lets a person click Freeze or Dismiss. The AI recommends and explains; the person decides, and every decision is logged.

In: the `scored`, `rings` and `cases` streams. **Out:** decisions on the `decisions` stream.

What the page shows

Area	Content	Read from
Top bar	Transfers scored, transfers flagged, open rings, p50 and p99 latency, updating live	`scored`
Ring list (left)	Newest and highest-scoring rings: ID, shape, number of accounts, XGBoost and GNN scores, first seen	`rings`

Area	Content	Read from
Ring picture (right)	Accounts as dots, transfers as arrows labelled with amounts; key account and collector highlighted	`rings`
Why	The top 3 reasons as plain phrases	`rings`
Summary	The case summary, with a small label: written by Granite (fact-checked) or by template	`cases`
Decision	Buttons: Freeze accounts, Dismiss, Needs more info; a note box	writes to `decisions`

How to build it with Streamlit

Streamlit turns a Python script into a web page; each button click reruns the script from the top.

1. Read the latest items with `XREVRANGE` (newest first, for example the last 200). The page doesn't need a consumer group, because it only reads.
2. Draw the ring with `st.graphviz_chart`: build the picture as Graphviz text (`digraph { "A" -> "B" [label="$9,800"] }`) and pass it in.
3. On a button click, `XADD` to `decisions`: `ring_id`, decision, note, time.
4. Refresh every 2 seconds: at the end of the script, `time.sleep(2)` then `st.rerun()`.
5. Run it with `streamlit run page.py --server.port 8501 --server.address 0.0.0.0` inside its container.

Check Streamlit at hour 0. It needs a library called pyarrow, which may not install on s390x. If `pip install streamlit` fails, switch straight away to **Flask**: one route that returns the latest data as JSON, and one HTML page that fetches it every 2 seconds. Same layout, about the same effort.

Seeing it from your laptop

Don't open ports to the internet. From your laptop:

```
ssh -L 8501:localhost:8501 linux1@<vm-ip>
```

Then open `http://localhost:8501`. Everyone on the team can do this at the same time.

Done when

- During a replay, the counters move, a demo ring appears with its picture, reasons and summary, and clicking Freeze writes a decision.
- The page never crashes when a stream is empty (start of a replay) or when a case hasn't arrived yet (show "summary coming").

LinuxONE platform

Owner: Pair 3. This is the highest-weighted judging category: the system must visibly run on LinuxONE, in containers, using the platform's strengths, with screenshots as proof. The VM is already set up with Docker and the official LinuxONE Jupyter image (which includes PyTorch, XGBoost and scikit-learn built for this machine).

The containers

Container	What runs in it	Image
redis	The queue (streams `txns`, `scored`, `rings`, `cases`, `decisions`)	`docker.io/library/redis:7`
replayer	Step 1	Our image (below)
scorer	Steps 2 and 3; can run as several copies	Our image
rings	Step 4, including the GNN	Our image
casewriter	Step 5, calls Granite	Our image
page	Step 6, on port 8501	Our image
notebook	Training and exploring (already running)	The LinuxONE Jupyter image

One image for all our services. Write one Dockerfile that starts `FROM` the LinuxONE Jupyter image, so PyTorch and XGBoost are already inside, then adds `redis` (and `streamlit` or `flask`) with pip and copies our code. Each service is the same image with a different start command. This avoids fighting s390x installs five times.

Docker Compose starts them together from one `docker-compose.yml`: one entry per service, all on the same internal network, all mounting `~/shared` as `/data`. Inside the network, services reach Redis at host name `redis`. Commands you'll use:

```
docker compose up -d --build       # build and start everything
docker compose ps                  # what's running (screenshot this)
docker compose logs -f scorer      # watch one service
docker compose up -d --scale scorer=3   # three scorers sharing the work
docker compose down                # stop everything
```

If the VM uses Podman instead of Docker, the same commands work with `podman` and `podman-compose`.

Order of work

1. **Hour 0 to 2:** Redis container up; `pip install redis streamlit` tested inside the Jupyter image (switch to Flask if Streamlit fails); confirm `nnpa`.
2. **By hour 8:** one transfer flows replayer to scorer to page, even with a dummy model.
3. **By hour 12:** the whole flow runs in Compose on the VM with the real model.
4. **Hours 12 to 16:** Telum comparison (see step 3), scaling demo, security settings, screenshots.
5. **Hour 16:** freeze. After this, only fixes.

What to show the judges, and how

Judging point	What we do	Screenshot or clip
Runs on LinuxONE	Everything on the VM	Terminal showing `uname -m` printing `s390x` and the VM's host name
Containerization	6 services in Compose	`docker compose ps` with all services up
Scalability	Scorers share the queue through a consumer group	Throughput with 1 scorer vs `--scale scorer=3`
Compute	Telum accelerator, if present	`nnpa` line from `/proc/cpuinfo`; Snap ML vs XGBoost timing table

Judging point	What we do	Screenshot or clip
Security	Redis has no published port; the page is reached only through an SSH tunnel; containers run as a non-root user (the Jupyter image's `jovyan`); API keys only in an untracked `.env`; data is synthetic	`docker compose ps` showing no Redis port; the `.gitignore`
AI on LinuxONE	XGBoost and the PyTorch GNN load and run inside containers on the VM	Logs showing model loading and live scores
Transparency	Every decision logged with time and reason	The `decisions` stream (`XRANGE decisions - +`)

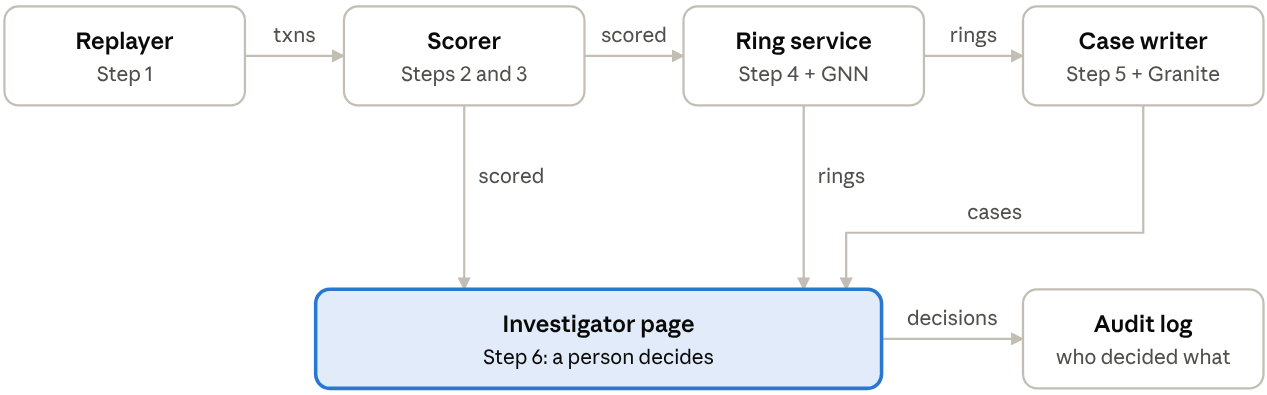
Optional extra (only if everything else is done)

IBM's Z Deep Learning Compiler (zDLC) can compile an ONNX model so it runs on the Telum accelerator. Export the GNN to ONNX with `torch.onnx.export` and try compiling it. Time-box this to 1 hour.

Handoffs between steps

Agree these names at hour 0 and don't change them without telling the whole team. If every pair sticks to them, each step can be built and tested alone, with fake messages, before the others are ready.

Every step talks only through Redis streams, so each can be tested alone



Each arrow is a Redis stream named on it; models and data files are shared through ~/shared.

services and streams · 6 services, 5 streams

The scorer, ring service and case writer each feed the page directly, so the page fills in step by step: scores first, then rings, then summaries.

Messages on the Redis streams

All values are sent as text. Lists and dictionaries are sent as JSON text (`json.dumps`) and read back with `json.loads` .

Stream	Written by	Read by	Fields
txns	replayer	scorer	txn_id, ts, from_acct, to_acct, amount_usd, amt_paid, ccy_paid, amt_recv, ccy_recv, fmt, sent_at
scored	scorer	rings, page	everything in txns, plus score, flagged (0 or 1), reasons (JSON list of 3 phrases), clues (JSON object of clue values), scored_at
rings	rings	casewriter, page	ring_id, version, shape, n_accounts, accounts (JSON list), edges (JSON list of txn_id, from, to, amount_usd, ts, score), xgb_score, gnn_score, reasons (JSON list), first_seen, last_seen

Stream	Written by	Read by	Fields
cases	casewriter	page	ring_id, version, summary (JSON object), source (granite or template), check_note, written_at
decisions	page	anyone, for the audit log	ring_id, decision (freeze, dismiss or more_info), note, decided_at

Files in ~/shared

File	Made by	Used by	Notes
data/clean.parquet	Pair 1	Everyone	Cleaned transfers, sorted, with txn_id and account IDs
data/splits.json	Pair 1	Everyone	The two cut-off timestamps
data/patterns.parquet	Pair 1	Pair 1 evaluation only	One row per pattern transfer, with attempt_id and shape
data/features.parquet	Pairs 1 and 2	Pair 2	One row per transfer: txn_id, label, every clue
models/feature_list.json	Pair 2	scorer, rings	The clue names **in the exact order** the model expects. A wrong order gives wrong scores with no error
models/scorer.json	Pair 2	scorer	XGBoost model, JSON only
models/thresholds.json	Pair 2	scorer, rings	alert and ring thresholds
models/gnn.pt or gnn.onnx	Pair 2	rings	GNN weights, trained on the VM, or ONNX if trained elsewhere

Test each step alone

Each pair writes a tiny script that pushes 5 fake messages in the format above onto its input stream, so it can test its step without waiting for the others. Keep these scripts; they are also how you debug on Sunday morning.

Demo video and submission

Owner: Pair 1, with everyone recording their part. The video is at most **3 minutes**, and going over costs points. It must cover the problem, the data, the solution, and how it uses IBM Z, including our tech stack and the LinuxONE instance and model running. Recording starts at hour 16 (around midnight Chicago time); aim to submit by 8 AM Sunday, an hour before building time ends.

A 3-minute script

Time	On screen	What we say
0:00 to 0:20	One slide: student M	The problem and the persona: students are recruited as money mules, and by the time a bank notices, the money is gone. Quote a sourced statistic, such as Cifas: 21 to 30 year olds were 42% of UK money mule activity in 2020.
0:20 to 0:35	Dataset slide	IBM's synthetic AML data: about 5 million transfers over 10 days, 1 in 981 laundering, so no real customer data is exposed.
0:35 to 0:55	Pipeline image	Six steps in one breath: stream, clues, score, ring, explain, decide.
0:55 to 2:00	Live screen recording on the VM	`uname -m` showing s390x; `docker compose ps`; start the replay; counters move; a ring appears; its picture, reasons and Granite summary; click Freeze.
2:00 to 2:35	Results and platform slide	Test numbers for XGBoost and the GNN; ring precision and time to alert; latency p50 and p99; Telum result; scaling to 3 scorers; security choices.
2:35 to 2:50	Impact slide	Who benefits (banks, investigators, and recruited students flagged as possible victims); the AI explains, a person decides.
2:50 to 3:00	Team slide	Six names and who built what.

Record the live part as one take with a screen recorder, then cut it down. Practise the voice-over twice with a timer before recording.

Submission checklist

- [ ] Project title exactly: **DevMaxx_IBM Z Datathon 2026 Project**
- [ ] Video uploaded, 3:00 or shorter, sound checked
- [ ] GitHub repo link; repo is public (or shared with the judges as instructed), with a README explaining how to run it
- [ ] README lists and links every open-source library and the dataset
- [ ] Pipeline image attached as the flowchart
- [ ] Screenshots folder in the repo (see **LinuxONE platform**)
- [ ] No API keys or `.env` in the repo (search the repo for "apikey" before submitting)
- [ ] Submitted on Hack2Skill before 9 AM Chicago time Sunday; confirm the exact deadline on Discord
- [ ] After the event: feedback survey by October 23

Numbers the pitch needs (Pair 1 collects them)

- [ ] Transfers in the dataset and the laundering rate (from your own count)
- [ ] XGBoost test precision, recall, F1, PR-AUC, and the $10,000 rule baseline
- [ ] GNN test precision and recall on the same transfers
- [ ] Ring results: share of known laundering attempts caught (a ring catches an attempt when at least half the ring's accounts belong to it), and how often our shape name matches
- [ ] Time to alert: minutes from an attempt's first transfer to its ring appearing
- [ ] Money still stoppable: share of an attempt's money that moved **after** the alert
- [ ] Latency p50 and p99; Telum vs CPU timing if available
- [ ] Granite fact-check pass rate
- [ ] One sourced statistic on young people recruited as money mules

When you're stuck

Try the fix in this table first. Still stuck after 20 minutes: tell the team chat what you tried, then ask in the Discord IBM expert channel (it opens on event day) with the exact error text.

Symptom	Likely cause	Fix
`pip install` compiles for ages or fails on the VM	No ready-made s390x package	Use what's already in the LinuxONE Jupyter image; build our image `FROM` it; run long installs inside `tmux`
`pip install streamlit` fails on pyarrow	pyarrow has no s390x build	Switch the page to Flask (step 6)
Model gives nonsense scores on the VM	Clues in a different order, or a pickled model	Use `feature_list.json` for the order; load `scorer.json`
Great validation scores, poor live scores	Train/serve skew or future data in clues	Run the 1,000-transfer leakage test from step 2
Precision near 0, recall near 1	Threshold too low, or `scale_pos_weight` far too high	Re-pick thresholds on validation; try the square root of the ratio
No rings appear	Ring threshold too high, window too short, or hubs not removed	Lower the threshold on validation, check the 24 h window, check the 300-counterparty hub rule
One giant ring	A hub account glues groups together	Apply the hub rule; split with Louvain
Services can't reach Redis	Using `localhost` inside a container	Use host name `redis`
Scorer at 100% CPU with no messages	`XREADGROUP` without `block=`	Add `block=1000`
`BUSYGROUP` error on start	The consumer group already exists	Catch that error and carry on
Granite reply fails the fact check every time	Prompt allows rounding or adds numbers	Lower temperature; give fewer numbers; widen the rounding tolerance a little; keep the template fallback
Granite call hangs	Network or key problem	Set a 15-second timeout, then use the template

Symptom	Likely cause	Fix
Page not reachable from laptop	No tunnel, or app bound to 127.0.0.1 inside the container	Use the SSH tunnel; start Streamlit with `--server.address 0.0.0.0` and publish port 8501 to `127.0.0.1` only
SSH drops and kills a job	Job tied to the terminal	Run long jobs in `tmux` (`tmux new -s train` , detach with Ctrl-b then d)
VM disk full	Docker images and data copies	`docker system prune` ; delete old parquet copies

Glossary

Term	Plain meaning
Money mule	Someone whose account is used to move stolen money, often a student recruited with a "easy money" offer
Ring	A group of accounts passing money between each other as part of one laundering attempt
Feature (clue)	A number describing a transfer that the model reads, such as minutes since money arrived
Label	The right answer for a row: 1 = laundering, 0 = normal
Imbalance	When one answer is very rare (here about 1 in 1,000), so accuracy is misleading
Precision	Of the transfers we flagged, the share that really were laundering
Recall	Of the real laundering transfers, the share we flagged
F1	One number balancing precision and recall (their harmonic mean)

Term	Plain meaning
PR-AUC	How good the model is across all thresholds, judged on precision and recall; the honest summary for rare events
Threshold	The score above which we act; chosen on validation
Time split	Training on earlier data and testing on later data, never shuffling
Leakage	When the model sees information it couldn't have in real life, such as future transfers or the label
Train/serve skew	When clues are computed differently in training and live, so live results are worse
XGBoost	A model made of many small decision trees, each correcting the previous ones
SHAP values	Per-clue contributions to one score; how we get the top 3 reasons
Union-find	A fast way to merge items into groups when pairs are linked; how we form rings
Louvain	A method that splits a big network into tightly connected communities
GNN	Graph neural network: a model that passes information along a network's links, so each transfer learns about its neighbourhood
Message passing	One GNN layer: each account gathers information from the transfers touching it
LLM	Large language model, a model that writes text; ours is IBM Granite
Fact check	Our rule that every number and account in Granite's summary must come from the verified facts
Redis Stream	A message queue in Redis that several programs can read in order

Term	Plain meaning
Consumer group	Redis feature that splits a stream's messages between several readers
Latency p50 / p99	The time within which 50% / 99% of transfers are scored
Container	A packaged program with everything it needs, run by Docker or Podman
LinuxONE	IBM's Linux server built on IBM Z mainframe hardware
s390x	The processor type of IBM Z and LinuxONE; software must be built for it
Big-endian	A byte order for storing numbers; LinuxONE uses it, laptops don't, which is why we save models as JSON or ONNX
Telum / NNPA	The AI accelerator built into the IBM Z processor chip; `nnpa` in `/proc/cpuinfo` means it's available
Snap ML	IBM's machine-learning library that can run tree models on Telum
ONNX	A standard file format for trained models that many tools can load
zDLC	IBM Z Deep Learning Compiler: turns an ONNX model into a program that can use Telum